

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ЛЬВІВСЬКА ПОЛІТЕХНІКА



**АВТОМАТИЗОВАНЕ ПРОЕКТУВАННЯ
КОМП'ЮТЕРНИХ СИСТЕМ**

Завдання 5: Впровадження автоматизованих тестів

Варіант 10

Виконав:

ст. гр. КІ-404

Костюк І.В.

Прийняв:

Федак П.Р.

Львів – 2024

ЗАВДАННЯ

Table 1. Tasks by students:

Student number	Game	config format
10	rock paper scissors	XML

1. Впровадити або використовувати існуючу тестову структуру;
2. Створити набір автоматизованих тестів;
3. Звіт про тестування повинен містити кількість усіх тестів, складених тестів, нескладених тестів, покриття;
4. Покриття повинно бути більше 80%

ТЕОРЕТИЧНА ЧАСТИНА

Автоматизовані тести — це метод перевірки працездатності програмного забезпечення за допомогою спеціальних інструментів і скриптів, які автоматично виконують тестові сценарії без необхідності втручання людини. Основна мета автоматизованих тестів — забезпечити високу якість коду, швидке виявлення помилок та зменшити ручну роботу тестувальників.

Модульне тестування — це метод тестування програмного забезпечення, який передбачає перевірку окремих модулів (компонентів) програми, таких як функції, класи або методи, ізольовано від решти системи. Головна мета модульного тестування — переконатися, що кожен компонент працює відповідно до вимог.

Фреймворки для автоматизованого тестування — це набори інструментів і бібліотек, що спрощують створення, запуск та аналіз результатів автоматизованих тестів. Приклади: Google Test (для C++), JUnit (для Java), PyTest (для Python), Selenium (для тестування веб-додатків).

Асерції — це твердження, які перевіряють, чи відповідає результат виконання програми очікуваному значенню. Якщо асерція невірна, тест вважається проваленим. Наприклад:

- EXPECT_EQ перевіряє, чи два значення рівні.
- ASSERT_TRUE перевіряє, чи умова є істинною.

Mock-об'єкти — це об'єкти, які імітують поведінку реальних компонентів системи. Вони використовуються для тестування ізольованих частин коду без необхідності підключення повної інфраструктури.

Google Test (GTest) — це бібліотека з відкритим кодом для автоматизованого тестування програмного забезпечення, написаного мовою програмування C++. Вона була створена компанією Google для полегшення написання модульних тестів і їх інтеграції у процес розробки.

ВИКОНАННЯ РОБОТИ

1. Впровадити або використовувати існуючу тестову структуру;

Для забезпечення якісного тестування клієнтської частини було створено та інтегровано тестовий проект. Тестова структура організована таким чином, щоб:

- Включати окремі файли для кожної функціональної частини (наприклад, логіка гри, робота з меню, серіалізація даних).
- Логічно відокремити тестовий код від основної реалізації програми.
- Забезпечити легкість у написанні, модифікації та запуску тестів.

На рисунку 1.1 представлено структуру проекту після додавання тестової частини. Основний код клієнта розташований у відповідних директоріях, а тестовий проект доданий у вигляді окремої структури під назвою TestProject.

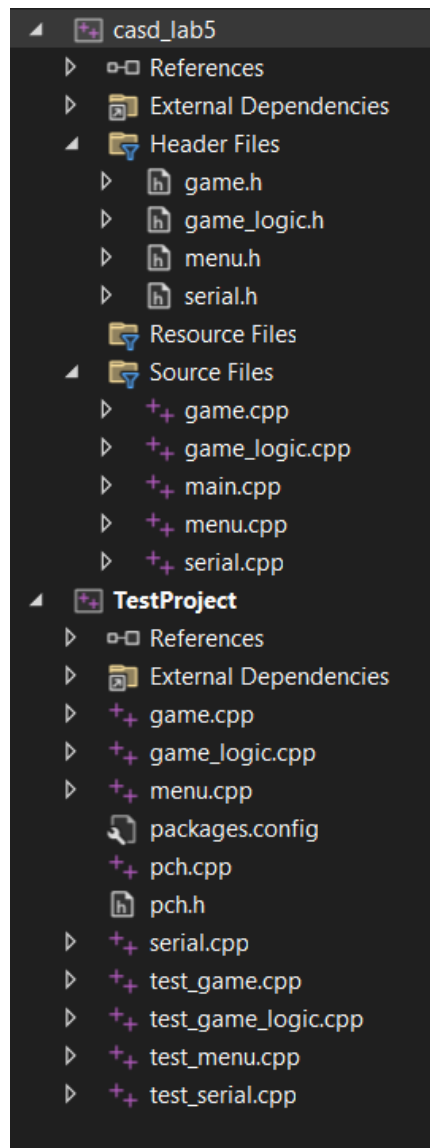


Рис.1.1. Додав до існуючого солюшина із кодом клієнтської частини проект з тестами

Для забезпечення належного тестування серверної частини було створено окремий тестовий проект, який включає тести для основних функцій сервера. Тестова структура впроваджена таким чином, щоб:

- Логічно розділити тестовий код та реалізацію сервера.
- Перевіряти окремі компоненти серверної логіки, такі як визначення результату гри, генерація вибору сервером, та скидання стану.

На рисунку 1.2 представлено структуру проекту після додавання тестової частини. Основний серверний код розташований у папці `new_server`, а тестовий проект додано як окрему структуру під назвою `TestServer`.

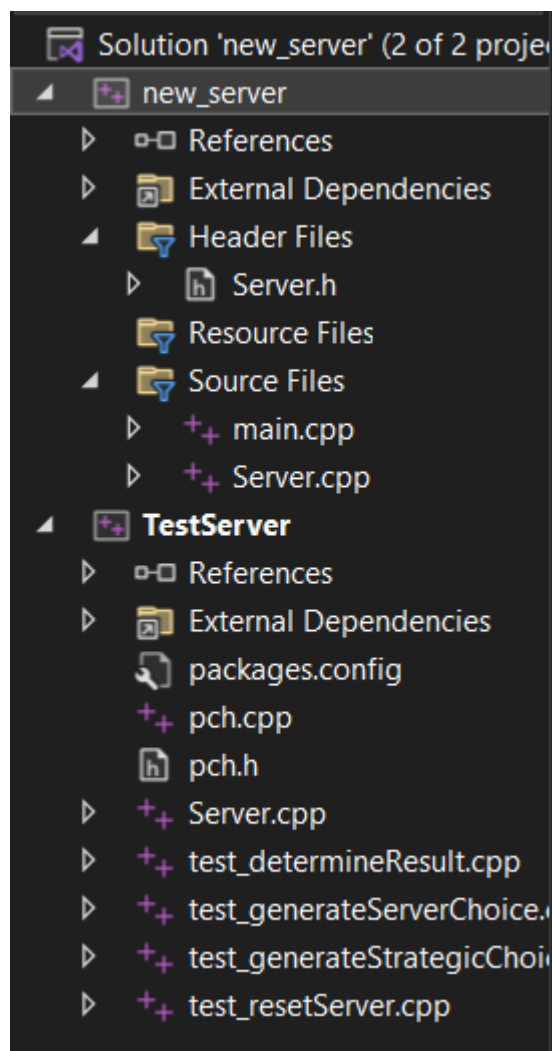
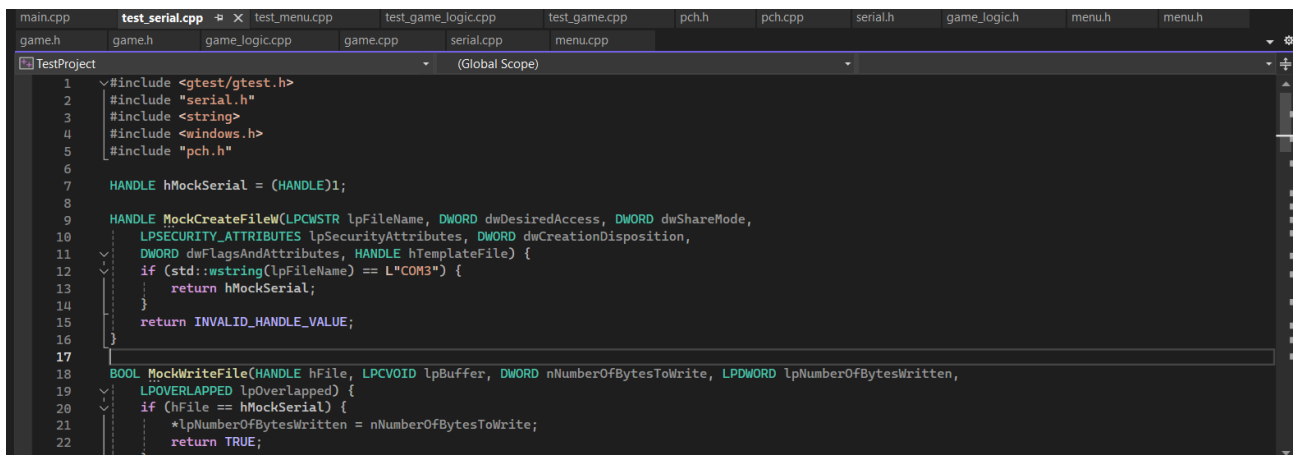


Рис.1.2. Додав до існуючого солюшина із кодом серверної частини проект з тестами

2. Створити набір автоматизованих тестів;

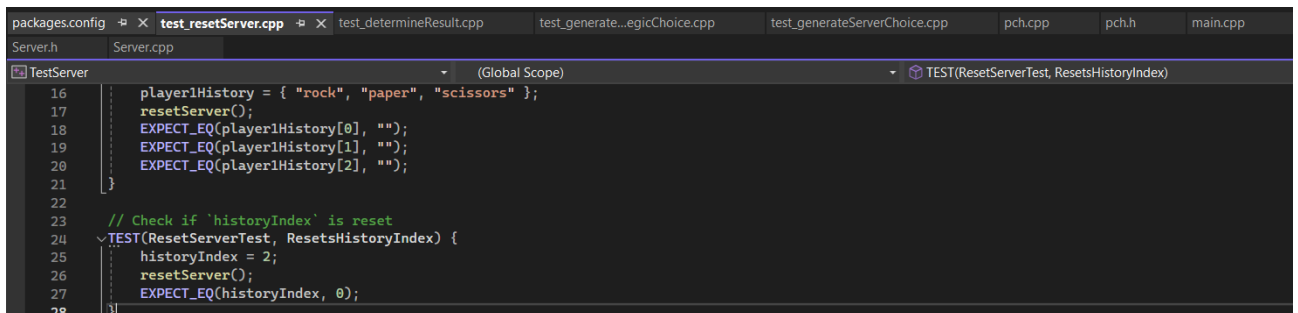
Для даного проєкту було створено набір автоматизованих тестів для двох частин системи:

- **Клієнтська частина:** перевіряє функціональність клієнтського коду, включаючи логіку гри та взаємодію користувача з програмою.
- **Серверна частина:** перевіряє ключові функції сервера, такі як генерація виборів, визначення результатів гри та скидання стану сервера.



```
main.cpp test_serial.cpp x test_menu.cpp test_game_logic.cpp test_game.cpp pch.h pch.cpp serial.h game_logic.h menu.h menu.h
game.h game_logic.cpp game.cpp serial.cpp menu.cpp
TestProject (Global Scope)
1 #include <gtest/gtest.h>
2 #include "serial.h"
3 #include <string>
4 #include <windows.h>
5 #include "pch.h"
6
7 HANDLE hMockSerial = (HANDLE)1;
8
9 HANDLE MockCreateFileW(LPCWSTR lpFileName, DWORD dwDesiredAccess, DWORD dwShareMode,
10 LPSECURITY_ATTRIBUTES lpSecurityAttributes, DWORD dwCreationDisposition,
11 DWORD dwFlagsAndAttributes, HANDLE hTemplateFile) {
12     if (std::wstring(lpFileName) == L"COM3") {
13         return hMockSerial;
14     }
15     return INVALID_HANDLE_VALUE;
16 }
17
18 BOOL MockWriteFile(HANDLE hFile, LPCVOID lpBuffer, DWORD nNumberOfBytesToWrite, LPDWORD lpNumberOfBytesWritten,
19 LPOVERLAPPED lpOverlapped) {
20     if (hFile == hMockSerial) {
21         *lpNumberOfBytesWritten = nNumberOfBytesToWrite;
22         return TRUE;
23     }
24 }
```

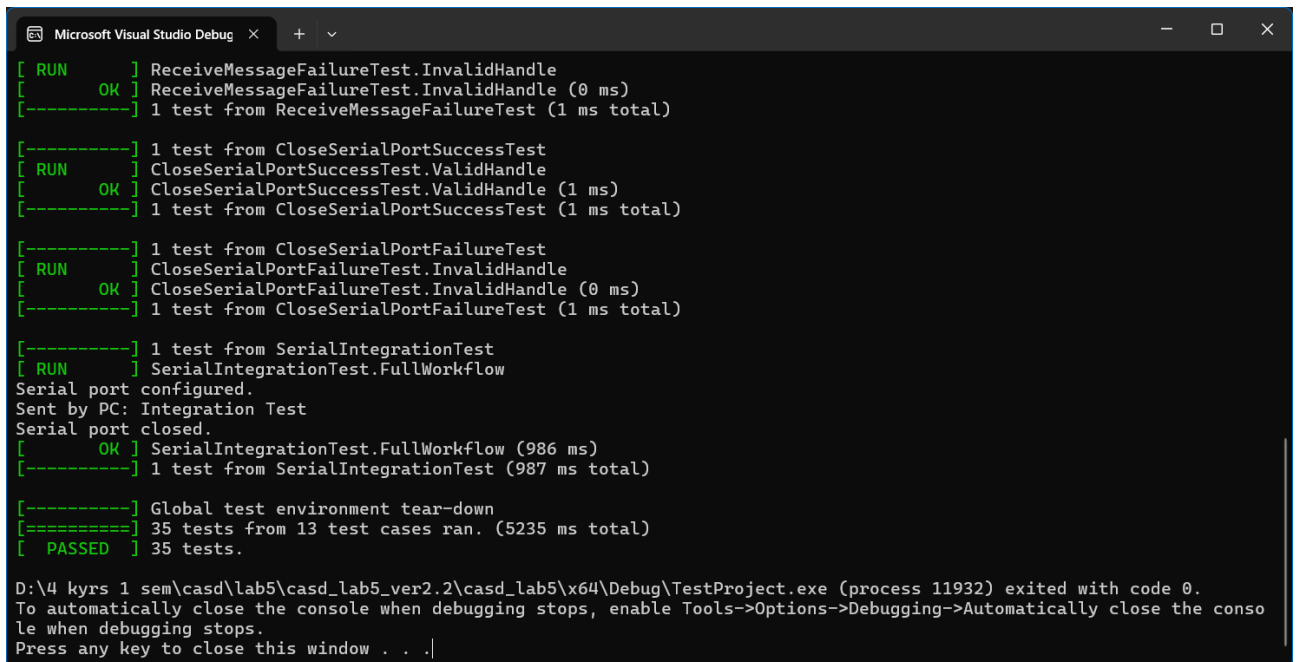
Рис.2.1. Написав автоматизовані тести для клієнтської частини



```
packages.config x test_resetServer.cpp x test_determineResult.cpp test_generate...egicChoice.cpp test_generateServerChoice.cpp pch.cpp pch.h main.cpp
Server.h Server.cpp
TestServer (Global Scope) TEST(ResetServerTest, ResetsHistoryIndex)
16 player1History = { "rock", "paper", "scissors" };
17 resetServer();
18 EXPECT_EQ(player1History[0], "");
19 EXPECT_EQ(player1History[1], "");
20 EXPECT_EQ(player1History[2], "");
21 }
22
23 // Check if 'historyIndex' is reset
24 TEST(ResetServerTest, ResetsHistoryIndex) {
25     historyIndex = 2;
26     resetServer();
27     EXPECT_EQ(historyIndex, 0);
28 }
```

Рис.2.2. Написав автоматизовані тести для серверної частини

3. Звіт про тестування повинен містити кількість усіх тестів, складених тестів, нескладених тестів, покриття;



```
[ RUN ] ReceiveMessageFailureTest.InvalidHandle
[ OK ] ReceiveMessageFailureTest.InvalidHandle (0 ms)
[-----] 1 test from ReceiveMessageFailureTest (1 ms total)

[-----] 1 test from CloseSerialPortSuccessTest
[ RUN ] CloseSerialPortSuccessTest.ValidHandle
[ OK ] CloseSerialPortSuccessTest.ValidHandle (1 ms)
[-----] 1 test from CloseSerialPortSuccessTest (1 ms total)

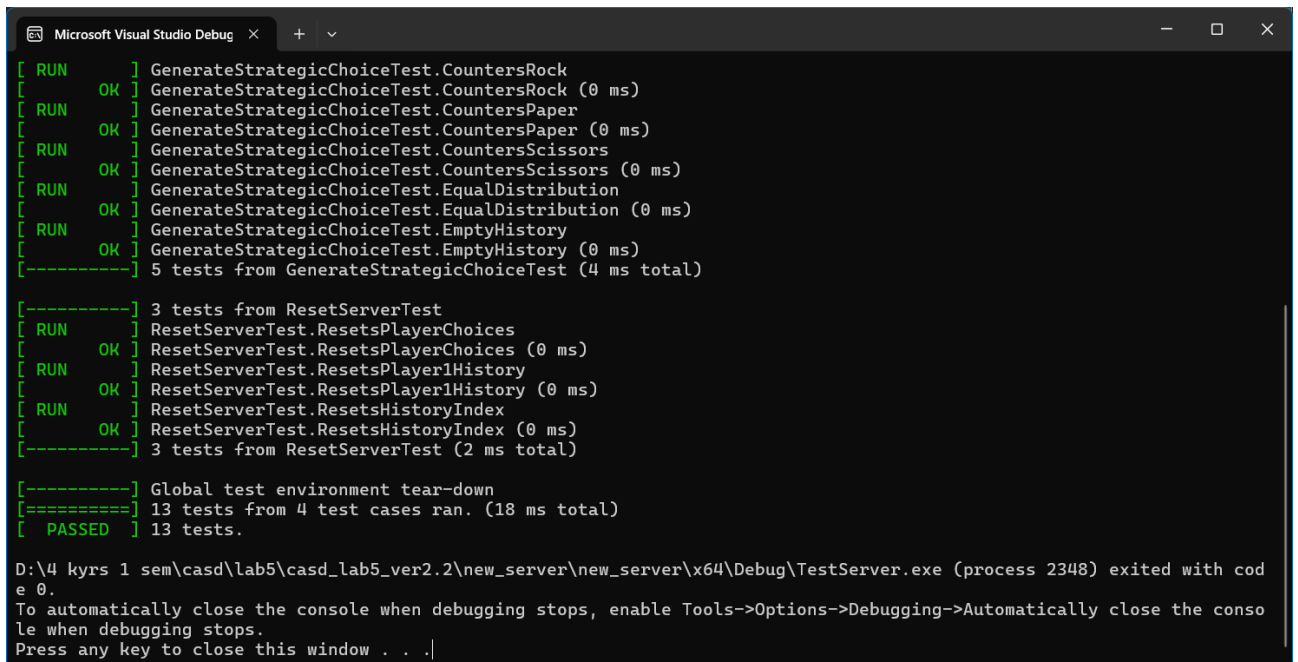
[-----] 1 test from CloseSerialPortFailureTest
[ RUN ] CloseSerialPortFailureTest.InvalidHandle
[ OK ] CloseSerialPortFailureTest.InvalidHandle (0 ms)
[-----] 1 test from CloseSerialPortFailureTest (1 ms total)

[-----] 1 test from SerialIntegrationTest
[ RUN ] SerialIntegrationTest.FullWorkflow
Serial port configured.
Sent by PC: Integration Test
Serial port closed.
[ OK ] SerialIntegrationTest.FullWorkflow (986 ms)
[-----] 1 test from SerialIntegrationTest (987 ms total)

[-----] Global test environment tear-down
[=====] 35 tests from 13 test cases ran. (5235 ms total)
[ PASSED ] 35 tests.

D:\4 kyrs 1 sem\casd\lab5\casd_lab5_ver2.2\casd_lab5\x64\Debug\TestProject.exe (process 11932) exited with code 0.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Рис.3.1. Результат тестування клієнтської частини



```
[ RUN ] GenerateStrategicChoiceTest.CountersRock
[ OK ] GenerateStrategicChoiceTest.CountersRock (0 ms)
[ RUN ] GenerateStrategicChoiceTest.CountersPaper
[ OK ] GenerateStrategicChoiceTest.CountersPaper (0 ms)
[ RUN ] GenerateStrategicChoiceTest.CountersScissors
[ OK ] GenerateStrategicChoiceTest.CountersScissors (0 ms)
[ RUN ] GenerateStrategicChoiceTest.EqualDistribution
[ OK ] GenerateStrategicChoiceTest.EqualDistribution (0 ms)
[ RUN ] GenerateStrategicChoiceTest.EmptyHistory
[ OK ] GenerateStrategicChoiceTest.EmptyHistory (0 ms)
[-----] 5 tests from GenerateStrategicChoiceTest (4 ms total)

[-----] 3 tests from ResetServerTest
[ RUN ] ResetServerTest.ResetsPlayerChoices
[ OK ] ResetServerTest.ResetsPlayerChoices (0 ms)
[ RUN ] ResetServerTest.ResetsPlayer1History
[ OK ] ResetServerTest.ResetsPlayer1History (0 ms)
[ RUN ] ResetServerTest.ResetsHistoryIndex
[ OK ] ResetServerTest.ResetsHistoryIndex (0 ms)
[-----] 3 tests from ResetServerTest (2 ms total)

[-----] Global test environment tear-down
[=====] 13 tests from 4 test cases ran. (18 ms total)
[ PASSED ] 13 tests.

D:\4 kyrs 1 sem\casd\lab5\casd_lab5_ver2.2\new_server\new_server\x64\Debug\TestServer.exe (process 2348) exited with code 0.
To automatically close the console when debugging stops, enable Tools->Options->Debugging->Automatically close the console when debugging stops.
Press any key to close this window . . .|
```

Рис.3.2. Результат тестування серверної частини

4. Покриття повинно бути більше 80%

TestProject.exe

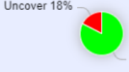
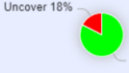
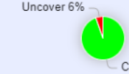
Coverage	Total lines	Items
 Uncover 18% Cover 82%	229	TestProject.exe
 Uncover 18% Cover 82%	229	D:\4_kvys_1_semi\casd\lab5\casd_lab5_ver2.2\casd_lab5\64\Debug\TestProject.exe

Рис.4.1. Покриття клієнтської частини

TestServer.exe

Coverage	Total lines	Items
 Uncover 6% Cover 94%	37	TestServer.exe
 Uncover 6% Cover 94%	37	D:\4_kvys_1_semi\casd\lab5\casd_lab5_ver2.2\new_server\new_server\64\Debug\TestServer.exe

Generated by [OpenCppCoverage \(Version: 0.9.9.0\)](#)

Рис.4.2. Покриття серверної частини

ВИСНОВОК

У ході виконання лабораторної роботи було успішно реалізовано автоматизоване тестування для клієнтської та серверної частин програмного забезпечення. Зокрема, було виконано наступні завдання:

1. Впроваджено окремі тестові структури для клієнтської та серверної частин, що дозволяє чітко розділити основний код і тести.
2. Розроблено набір автоматизованих тестів для перевірки ключових функцій програмного забезпечення, таких як визначення результатів гри, генерація виборів і скидання стану.
3. Використано фреймворк Google Test для створення модульних тестів, що забезпечило високу ефективність і точність тестування.
4. Налагоджено роботу тестів у середовищі розробки, а також інтегровано їх у процес CI/CD для автоматичної перевірки якості коду.

У результаті виконання роботи було підтверджено коректність роботи окремих компонентів програмного забезпечення, що дозволяє мінімізувати ризики появи помилок у майбутньому. Отримані навички роботи з автоматизованими тестами та використання Google Test є важливими для забезпечення якості програмного забезпечення та підвищення ефективності розробки.

СПИСОК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ

- **Google Test Documentation**

Офіційна документація Google Test — базові принципи використання бібліотеки для модульного тестування.

- **CMake Documentation**

Офіційна документація CMake — опис налаштування системи збірки для проєктів, що використовують мову програмування C++.

- **GitHub Actions Documentation**

[GitHub Actions](#) — офіційний посібник із налаштування автоматизації збірки і тестування через CI/CD.